

Lab 03

Practical Introduction to Large Language Models

Gianluca Aguzzi¹ Mirko Viroli¹

¹Alma Mater Studiorum – Università di Bologna
{gianluca.aguzzi, mirko.viroli}@unibo.it

March 5, 2026

One slide sum-up on LLM

Large Language Models

- State of the art in generative AI for text processing and generation
- Paradigm shift – foundational models that can be adapted to many tasks and domains
- Adaptation through fine-tuning (not covered) and prompt-engineering (few-shot learning, zero-shot, etc.)
- Many models available with different characteristics

LLM in Software Engineering: Why we care

- They may transform the software engineering landscape toward Software Engineering 3.0
- As software engineers, we should be able to use them, understand them, and possibly adapt them
- How to create applications with LLMs? How to test them?
 - ▶ Today we will focus on the first, and we will have a glimpse on the second

Starting point and goals

References

- Ollama: <https://ollama.com/>
- LangChain4J: <https://github.com/langchain4j/langchain4j>
- 03-LLM-intro: <https://github.com/cric96/asmd-public-03-llm-intro-lab>
- LLMs: <https://ollama.com/search> looks for small models (less than 1B parameters, at most 3B)
- Slide 03 provide few examples of prompt-engineering

General goals

- be operative with Ollama and langchain4j
- experiment with prompt-engineering
- pre-check on how LLMs can be used to create software applications

Operational Steps: Play with LLMs

Step 1 – Install Ollama

- Download and install Ollama from <https://ollama.com/> (or use Docker)
- Install required models (see the README of the repo for instructions)

Step 2 – Set Up Project

- Clone repository: <https://github.com/cric96/asmd-public-03-llm-intro-lab>
- Verify everything inside `it.unibo.basics` works fine by running without errors

Step 3 – Understand the Codebase

- Examine the code in the `it.unibo.basics` package, which contains basic utilities for interacting with Ollama
- Try to modify models and parameters to observe different results
- Look at the `prompt` package, which contains examples of prompt engineering
- Try to create your own prompt and see how it performs (e.g., ask for code generation or explanations of code)
 - ▶ Also experiment with few-shot prompting by providing examples in the prompt
- Finally, look at the `tictactoe` package, which contains a simple application that uses LLMs to play tic-tac-toe
 - ▶ Focus in particular on the `controller` package and the `formatter`, `parser`, and `prompt` packages

Step 4 – Experiment with Prompt Engineering

- Look at the `e1` package, which contains an application that uses LLMs to create a role-play text-based game
- Read the `e1/EXERCISES.md` file, which contains the steps to experiment with prompt engineering
 - ▶ First, implement the `Advance` and `Begin` prompts
 - ▶ Second, unit test the logic of `LLMStoryEngine` by mocking the LLM interactions

Tasks

MEMORY MANAGEMENT

Goal: Add memory management capabilities to the story engine to achieve long-term narrative coherence across LLM calls.

Primary Task: Design and implement a prompt decorator that maintains a summary of previous story beats and injects it into each new request, compensating for the stateless nature of LLM interactions.

Advanced Task: Investigate different context management strategies (e.g., fixed-window history vs. LLM-based summarisation) and analyse the trade-offs between narrative coherence, context length, and token cost.

LLMs VERIFICATION

Goal: LLM outputs are non-deterministic — the same prompt can yield different results depending on temperature, sampling, and model state. Explore how to systematically test and validate LLM-powered components despite this inherent variability.

Primary Task: Design a testing strategy for the story engine that accounts for non-determinism. Experiment with temperature settings to observe their effect on output stability, and implement checks that tolerate variability while still asserting correctness (e.g., a second LLM acting as a judge).

Advanced Task: Reflect on what “correctness” means for generative outputs and how confidence and reproducibility relate to temperature. Investigate whether lowering temperature is sufficient for reliable testing.

AI-APP DESIGN

Goal: Integrate an LLM into one of your existing applications or projects, replacing static rules or simple AI logic with a language-model-driven component.

Primary Task: Identify a decision point in a past project where an LLM could add value, implement the integration, and reflect on the design choices and limitations introduced by the non-deterministic nature of the model.